

Social Media Post Feature Specification

Architecture, Clean Code, API Integration, and Workflows

Status: Production Ready	Version: 1.0.0	Date: October 2026
Client: Flutter / Riverpod	Backend: Rust / Axum Microservices	Tests: 56 / 56 Passing

1. Executive Summary

The Lotus Connect social post feature delivers a modern Facebook and Instagram inspired feed experience. It supports rich post authorship (multi-media uploads, gradient status cards, feelings, geo-tags, privacy filters), seamless in-place post editing with attachment management, and secure post deletion backed by native confirmation dialogs and optimistic feed updates.

Key Functional Pillars:

- **Post Creation:** Multi-image/video file picking, automated upload via Gateway, colored background card fallbacks, and real-time feed prepending.
- **Post Editing:** Pre-filled edit form, editing existing network attachments, adding new media, and in-place feed updates.
- **Post Deletion:** Destructive action with safety dialog and backend verification of author ownership.
- **Feed Interaction:** Double-tap animated heart likes, bookmarking, clipboard link sharing, and report/hide options.

2. System Architecture & Layer Breakdown

The architecture adheres strictly to Clean Architecture principles, ensuring distinct separation between Presentation, Application (State), Domain, Data, and Backend services.

Layer	Key Components	Responsibilities
Presentation	HomeScreen, PostCard, CreatePostScreen, PostOptionsSheet, PostPrivacySheet	Renders UI, handles touch feedback & haptics, presents sheets & confirmation dialogs.
Application	FeedNotifier, CreatePostNotifier, SettingsNotifier	Riverpod StateNotifiers managing feed items, edit draft state, and current user identity.
Domain	CreatePostUseCase, UpdatePostUseCase, DeletePostUseCase, GetFeedUseCase	Pure business logic, automated media upload coordination, and parameter validation.
Data	FeedRepositoryImpl, FeedRemoteDataSource, DioClient	REST HTTP transport, multi-part uploads, JWT authentication interceptors, JSON serialization.
Backend	Axum Gateway, Feed Service, MinIO Storage, PostgreSQL	Microservice routing, author authorization checks, transaction persistence, and media hosting.

3. Integration Workflows & Execution Sequences

3.1 Post Creation Workflow

1. **User Action:** User inputs text in CreatePostScreen, optionally selects media files or feeling.
2. **Media Upload:** If local files are chosen, CreatePostUseCase calls FeedRepository.uploadFiles via POST /api/v1/media/upload.
3. **Persistence:** CreatePostUseCase posts the payload to POST /api/v1/posts. Gateway routes to Feed Service.
4. **Feed Update:** FeedNotifier.addPost(newPost) prepends the new post directly to the live feed.
5. **Feedback:** Screen triggers haptic feedback, shows a green confirmation toast, and navigates back.

3.2 Post Editing Workflow

1. **Initiation:** User taps '...' on PostCard. PostOptionsSheet displays author actions.
2. **State Hydration:** CreatePostScreen opens with postToEdit. CreatePostNotifier.initializeForEdit sets isEditMode=true and loads existing content & media.
3. **Media Merging:** User can remove existing media chips or pick new local files.
4. **Backend Update:** UpdatePostUseCase uploads any new local files, merges with existing remote items, and sends PUT /api/v1/posts/:id.
5. **Live Sync:** FeedNotifier.updatePost(updatedPost) replaces the post item in place within Riverpod feed state.

3.3 Post Deletion Workflow

1. **Initiation:** User taps 'Delete post' (highlighted in destructive red) in PostOptionsSheet.
2. **Safety Confirmation:** An AlertDialog prompts: 'Are you sure you want to delete this post? This cannot be undone.'
3. **API Invocation:** Upon confirmation, DeletePostUseCase executes DELETE /api/v1/posts/:postId.
4. **Security Check:** Feed Service validates author_id == user_id and soft-deletes/removes record.
5. **Feed Pruning:** FeedNotifier.removePost(postId) removes the post from the UI instantly.

4. Backend API Specifications

Method & Endpoint	Auth	Payload / Params	Success Response
POST /api/v1/posts	Bearer JWT	content, mediaItems[], visibility, feeling, location	201 Created PostItem JSON
PUT /api/v1/posts/:id	Bearer JWT	content, mediaItems[], visibility	200 OK Updated PostItem JSON
DELETE /api/v1/posts/:id	Bearer JWT	Path param: post_id	200 OK {success: true}
POST /api/v1/media/upload	Bearer JWT	Multipart form-data: files[]	200 OK List of PostMediaItem
GET /api/v1/feed	Bearer JWT	Query params: page, limit	200 OK List of PostItem

5. Security, Ownership & Moderation

Permissions are enforced symmetrically on both client UI and backend database handlers.

Role	Client Options Sheet	Backend Authorization Rule
Author (user.id == post.author.id)	• Edit post • Edit audience (Public, Friends, Private) • Delete post (Destructive alert) • Copy link	Full read/write/delete permissions. Validated against JWT sub claim.
Viewer (user.id != post.author.id)	• Save post (Bookmarks) • Hide post (Client suppression) • Report post (Moderation trigger) • Copy link	Read-only access. Modification requests strictly rejected with HTTP 403 Forbidden.

6. Quality Assurance & Test Verification

All features are validated with automated test suites across domain use cases, state notifiers, and presentation widgets:

- **UpdatePostUseCaseTest:** Validates input parameters, upload delegation, and media merging.
- **DeletePostUseCaseTest:** Validates post ID checks and repository delegation.
- **CreatePostNotifierTest:** Verifies edit state initialization, text modification, and feed synchronization.
- **FeedNotifierTest:** Tests in-place post updates and instant removal on delete.
- **PostOptionsSheetTest:** Asserts author vs viewer options, confirmation dialog, and destructive callbacks.
- **CreatePostScreenTest:** Asserts Edit post UI rendering, prefilled text, and Save button dispatch.

Automated Test Suite Status: 56 / 56 Tests Passed (100%)
Static Analysis Status: 0 Errors, 0 Warnings (flutter analyze clean)